

BEACON LABS

CYBERSECURITY EDUCATION LABS

Off-Path TCP Exploits Lab

Module: Network Security



BOISE STATE UNIVERSITY

COLLEGE OF ENGINEERING

Department of Computer Science

Table of Contents

Overview	5
Virtual Machine Setup	6
Task Set 1: Clock Synchronization	7
Task 1.1: Initiate a Legitimate TCP Connection.....	7
Task 1.2: Send 200 Packets Equally Spaced.....	8
Task 1.3: Adjust Time Delay	9
Add The Variable of n_result.....	10
Task Set 2: Four-Tuple Inference	11
Task 2.1: Send SYN-ACK Packets	12
Task Set 3: Sequence Number Inference & Nash Equilibrium	13
Task 3.1: Determine the Correct Sequence Number	13
Task 3.2: Narrowing Down the Search Space.....	14
Task 3.3: Find the Sequence Number.....	15
Task Set 4: Acknowledgement Number Inference.....	16
Task 4.1: Determine the Correct Acknowledgement Number	16
Submission	18
Summary.....	18
References	18

Terminology

ACK-window The ACK-window, or Acknowledgement window, is a concept in TCP (Transmission Control Protocol) that represents the range of acceptable sequence numbers that are acknowledged by the receiving end of a TCP connection.

Blind In-Window Attacks Blind in-window attacks are TCP attacks in which the attacker isn't in the path of data transfer but is able to pose as one side of the TCP exchange by sending packets externally that are "in-window" of the expected packets.

Challenge-ACK Challenge Acknowledgement (Challenge-ACK) is a security feature in the TCP protocol that aims to protect against blind in-window attacks.

Global Rate Limit A mechanism to prevent resource exhaustion attacks by putting a limit on the number of responses (i.e., ACKs) that a server can send.

Acknowledgement Number Inference A technique that is used in network security, particularly in TCP/IP hacking. It entails inferring the sequence of numbers that the receiving end of a TCP connection is expecting next (the ACK numbers) based on observing the pattern of previous ACK numbers.

Overview

Objective

Understand the flow of an off-path TCP exploit and gain the ability to execute an off-path TCP exploit through completing the essential code to exploit the Global Rate Limit vulnerability.

Off-Path TCP Exploits

Off-Path TCP Exploits refer to a class of cybersecurity attacks where an attacker can manipulate or infer data from a TCP session between two other parties without being in the data transmission path.

In Linux Kernel version 3.6, RFC (Request For Comment) 5961 was faithfully implemented to stop blind in-window attacks but, it also created a new vulnerability. This vulnerability lies in the Global Rate Limit, which limits the number of challenge-ACKs to 100 per second to prevent excessive resource usage. Figure 1 demonstrates this below.

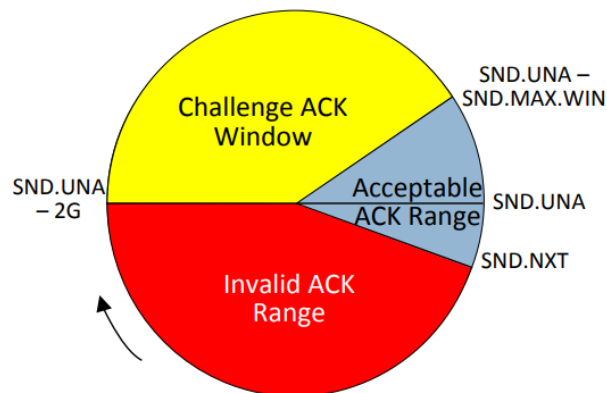


Figure 1: ACK Window Illustration

The Global Rate Limit can be exploited to show if a TCP connection is present and, if a TCP connection is found, the Global Rate Limit can then subsequently be used to infer the four-tuple of the client and server, the next acceptable sequence number, and the acknowledgment number. After inference of these three components is completed, it is trivial for an attacker to inject spoofed packets.

Resources

Video explaining how the attack works:

<https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/cao>

Paper that the attack is based upon:

https://www.cs.ucr.edu/~zhiyunq/pub/sec16_TCP_pure_offpath.pdf

[1]

Virtual Machine Setup

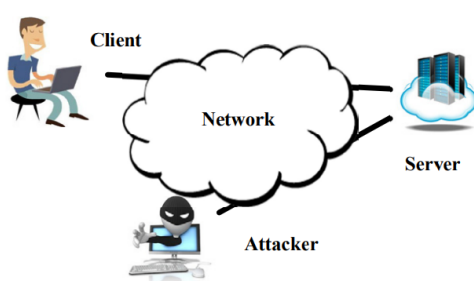
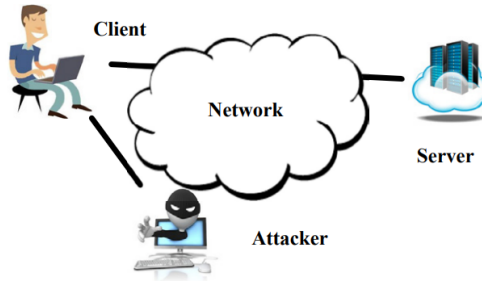
In this lab, three virtual machines will be needed: the client, server, and attacker. There are only two virtual machine images that need to be downloaded since the client and attacker use the same image. The virtual machines will be referred to as the "Client VM," "Server VM," and "Attacker VM."

A Python script is used to create a simple messaging service between the client and server, thereby creating the TCP connection that will eventually be hijacked.

The server is running a version of Ubuntu 14.04 that has not been updated. The security updates for this operating system serve to patch the vulnerability being exploited, so it's crucial to avoid any updates. Both the client and attacker are running Ubuntu 16.04 with the libtins libraries installed. These libraries are used to spoof and send packets, so they are necessary to have on the virtual machine for the attacker to utilize.

By interacting with the server as seen in Figure 2 on page 7, all steps of the attack can be completed. After this is completed, it then becomes possible to inject packets imitating the server to the client, as seen Figure 3 on page 7.

For setup instructions, see the provided Setup document.

*Figure 2: Exploiting Server**Figure 3: Imitating the Server*

Task Set 1: Clock Synchronization

When working with the clock throughout this lab, it's imperative that we tell the program to wait until the clock hits a certain time rather than telling it to sleep for a certain period of time. Keep this in mind while completing the lab.

Objective

Synchronize the Attacker VM's clock with the server clock on the Server VM such that all the packets sent from the Attacker arrive within the proper time interval.

Task 1.1: Initiate a Legitimate TCP Connection

A legitimate TCP connection first needs to be established by sending the appropriate flags to the server.

Begin by going to the Attacker VM and opening a terminal. Navigate to the "~/Documents/attacker_cpp" directory and then open the C++ file that needs to be edited with the following command:

```
$ gedit synchronize_clock.cpp
```

Action

- 1.1.1 - Choose the correct TCP flags to initiate the TCP connection.
 - 1.1.2 - Write the flag to initialize the TCP connection on line 25 of `synchronize_clock.cpp`.
 - 1.1.3 - On line 47, disable the flag you set on line 25.
 - 1.1.4 - On line 48, set the flag that indicates that the response from the server has been received.
- (Figure 4, page 8).

```
25 legit_TCP.set_flag(TCP::Flags::/* Delete this comment write code here */, 1);
26
27 std::unique_ptr <PDU> response(sender.send_recv(legit_IP));
28
29 if (!response) {
30     std::cout << "No response!" << std::endl;
31 }
32
33 TCP &legit_TCP_response = response->rfind_pdu<TCP>();
34 if (!(legit_TCP_response.get_flag(TCP::SYN) &&
35     legit_TCP_response.get_flag(TCP::ACK))) {
36     std::cout << "Didn't respond with SYN ACK!\n";
37     std::cout << "Check given server PORT...\n";
38     exit(-1);
39 } else {
40     std::cout << "connection established\n";
41 }
42 uint32_t seq_response = legit_TCP_response.seq();
43 //increment ack seq
44 legit_TCP.ack_seq(seq_response + 1);
45 legit_TCP.seq(1);
46
47 legit_TCP.set_flag(TCP::Flags::/* Delete this comment write code here */, 0);
48 legit_TCP.set_flag(TCP::Flags::/* Delete this comment write code here */, 1);
```

Figure 4: `synchronize_clock.cpp` Step 1

Task 1.2: Send 200 Packets Equally Spaced

The next step in synchronizing the clock is to send 200 packets that are equally spaced in the span of one second to see how many challenge-acks this prompts. Since only 100 challenges can be sent within a second, it can be concluded that the clock is synchronized when only 100 challenges are received.

There is a chance that the clock will already be synchronized, but it is far more likely that more than 100 challenges will be received. This process is attempted twice, adjusting the time between the first and second attempts. The third attempt uses the calculated time delay necessary to receive only 100 challenges.

Action

1.2.1 - Write the necessary line of code to equally space out the 200 packets in the span of one second so that each packet has an equal amount of time between the previous and next packets on line 75 of `synchronize_clock.cpp` for round one as seen in Figure 5 below.

```
71
72     for (int i = 1; i <= 200; ++i) {
73         sender.send(eth);
74
75         // WRITE CODE HERE:
76
77
78     }
79
```

Figure 5: `synchronize_clock.cpp` Step 2

Task 1.3: Adjust Time Delay

Prior to beginning round two, we need to shift the time that we start sending packets by 5 milliseconds. After the 5 milliseconds is up, we must perform the same process of sending out 200 equally spaced packets in the span of one second and then checking the number of challenges we received back. Just like in round 1, if we received 100 challenges then our clock is now synchronized and if we received more than 100 challenges, we need to continue on with round 3.

Action

1.3.1 - Write your code on line 98 for round two as seen in Figure 6 on page 10 below.


```

93
94 // Round 2
95 for (int i = 1; i <= 200; ++i) {
96     sender.send(eth);
97
98     // WRITE CODE HERE:
99
100
101 }
102

```

Figure 6: *synchronize_clock.cpp* Step 3

Task 1.4: Add the variable of `n_result`

If round 3 has been reached, that means that we have more information to work with. Specifically, we can compare the number of packets received in round 1 to the number of packets received in round 2.

Let's assume that in round 1, x RST packets were sent to the server over the first second and in the second second, y RST packets were sent to the server. Note that $x + y = 200$. In round 2, we can assume that $(x - 1)$ RST packets were sent to the server in the first second and $(y + 1)$ RST packets were sent to the server in the second second due to the attacker waiting those 5 milliseconds before starting round 2. Therefore, we can calculate the number of packets from round 1 and round 2 as:

$$n_1 = \min(x, 100) + \min(y, 100)$$

$$n_2 = \min(x - 1, 100) + \min(y + 1, 100)$$

Therefore, if $n_2 \geq n_1$ then we can assume that $y \geq 100$ and $x \leq 100$ which means that $n_1 = x + 100$ and $n_2 = (x - 1) + 100 < n_1$ which contradicts $n_2 \geq n_1$. Therefore, $y < 100$ and $x > 100$ which means that $n_2 = 100 + (y + 1) = 100 + (200 - x + 1)$ which can be simplified to: $(x - 1) = 300 - n_2$. From this, the attacker will know that in the event that $n_2 \geq n_1$, they will need to wait $(300 - n_2) \cdot \frac{1}{200}$ seconds to synchronize their clock with the server's clock.

Likewise, if $n_1 > n_2$ then the attacker will know that $y > 100$ and $x < 100$ which means that $n_2 = (x - 1) + 100$ and therefore the attacker will need to wait $\frac{n_2 - 100}{200}$ seconds to synchronize their clock with the server's clock.

Note: in the code you'll see the calculation is slightly different. This is because the conversion from seconds to milliseconds has been done and simplified. I.e., $\frac{n_2-100}{200} \cdot 1000 = (n_2 - 100) * 5$

Action

1.4.1 - Write your code on line 127 for round three as seen in Figure 7 below.

```
122
123     for (int i = 1; i <= 200; ++i) {
124         sender.send(eth);
125
126
127         // WRITE CODE HERE:
128
129     }
```

Figure 7: *synchronize_clock.cpp* Step 4

After altering and saving the code, compile it by running the following command:

```
$ rm -f exploit && make
```

You can then run the partial exploit by executing the following command:

```
$ sudo ./exploit <client_ip> <server_ip> <server_port>
```

If all tasks in this task set are done properly, you should see "CLOCK SYNCHRONIZATION COMPLETE" appear as output in the terminal.

There is a redundancy in the code to try the time synchronization again in case there was any packet loss. If the code is written correctly by the student it should only take one round to complete but there is a possibility it will need a second or third.

Task Set 2: Four-Tuple Inference

Objective

Infer the four-tuple of the client and server to then utilize the four-tuple in the identification of the correct client port.

This task set will be conducted in the `4tuple_inference.cpp` file.

Use the following command to open the correct file:

```
$ gedit 4tuple_inference.cpp
```

Task 2.1: Send SYN-ACK Packets

Over the course of a second, SYN-ACK packets are sent out to the server with the four-tuples of `<srcIP = clientIP, dstIP = serverIP, srcPort = X, dstPort = serverPort>`. The source port is labeled as X as it is the part that needs to be figured out. The default Linux port range is 32768 to 61000. By halving this and sending packets to ports in the range 46884 - 61000, it's possible to determine which part of this range contains the correct port number.

Once SYN-ACK packets have been sent out in the first half, 100 RST packets can be sent in the same second. If 100 challenge-ACKs are observed, it can be inferred that it's in the first half of the search space. If 99 are observed, it can be inferred that one was sent to the client, and thus the correct port is in the second half. The observed amount is used to determine if the correct client port is within the searched range and how the right or left port should be adjusted accordingly.

Action

2.1.1 - In `'4tuple_inference.h'`, update the port constants to accurately reflect the default Linux port range.

2.1.2 - Remove lines 12-16 and lines 119-128, inclusive.

2.1.3 - Complete the binary search algorithm by writing your code on line 67 as seen in Figure 8 on page 13 below.

After altering and saving the code, compile it by following the same steps as described previously. If the code is written correctly, the terminal should read `"Client port:#####" where "#####" is the same as the "client port" shown in the Server VM's terminal. After finding the client port, the next step is to locate the sequence number.`

```
62     }
63     // waiting to make sure all packets can be received
64     std::this_thread::sleep_until(clock);
65
66
67     // WRITE CODE BELOW:
68
69
70
71
```

Figure 8: *4tuple_inference.cpp* Step 1

Task Set 3: Sequence Number Inference & Nash Equilibrium

Objective

Infer the sequence number such that the Attacker's packets are received in the desired order.

This task set will be conducted in the `sequence_inference.cpp` file. Use the following command to open the correct file:

```
$ gedit sequence_inference.cpp
```

Task 3.1: Determine the Correct Sequence Number

First, it is necessary to determine the correct sequence number. This is done by sending packets to each block of the sequence number space and observing the server's response.

Action

- 3.1.1 - Remove lines 21-26, 55-59, 62, 63, and 67 inclusive.
- 3.1.2 - Analyze the game theory table provided below in Table 1 on page 14 and decide whether to search high or low first.
- 3.1.3 - Explain your decision in your lab report.
- 3.1.4 - Complete the for-loop conditions (initialization, condition, and increment/decrement) on line 61 of 'sequence_inference.cpp' to reflect your decision.

		Defender	
		Start Low	Start High
Attacker	Search Low	(2, -2)	(2, -2)
	Search High	(1, -1)	(3, -3)

Table 1: Game theory table representing the advantages and disadvantages of both (the attacker and the defender)

The defender has the option of starting the sequence number with a high value or a low value. Starting with a high value costs more resources, so it gives the attacker 2 and the defender -2. If the defender starts it at a low value, then it's 1 and -1 instead. If the attacker chooses correctly to search high or low first, then another 1 and -1 are given to the attacker and defender, respectively.

Task 3.2: Narrowing Down the Search Space

The next step in the exploit is to narrow the search space down to a single block. This is done through another binary search in which we observe how many packets are received after sending them to one half of the search space at a time.

The definitions of the variables are as follows:

SEQ_MAX is the maximum value a 32-bit integer can be.

WINDOW_SIZE is 12,600 as that is the default window size for Linux.

chunk_size is 8,000 which is the area where a packet is sent to each round.

SEQ_MAX/chunk_size/WINDOW_SIZE is the maximum chunk_id to be searched.

Task 3.3: Find the Sequence Number

We can find the correct sequence number through a three step process:

1. Approximate the range in which the number lies.
2. Find the correct block.
3. Find the sequence number itself.

The search space is narrowed down to a single block through another binary search algorithm, similarly to the 4-tuple inference. The search space is then progressively narrowed down by observing the number of packets received after sending packets to one half of the space. You will **not** have to write any code for this step.

The iteration through sequence numbers will continue until the correct sequence number is found. Once the correct sequence number is found, we'll infer the next acknowledgement number. You will **not** have to write any code for this step either.

Complete the final step of this task by doing the following:

Action

3.3.1 - Write the code to send packets on the legitimate connection to exhaust the global rate limit on line 73 of 'sequence_inference.cpp'.

3.3.2 - Create an if-statement to check if the correct block is in the current search space on line 91 of sequence_inference.cpp as seen in Figure 9 on page 16 below.

After altering and saving the code, compile it using the same process as before.

```
66 // Sending spoofed packets to test where the sequence number is
67 for (uint32_t block_i = 0; block_i < chunk_size; block_i++) {
68     spoofed_TCP.seq((block_i * WINDOW_SIZE) + chunk_id * chunk_size * WINDOW_SIZE);
69     sender.send(spoofed_IP);
70 }
71
72
73 // WRITE CODE HERE:
74
75
76
77
78 auto finish = std::chrono::high_resolution_clock::now();
79 clock += std::chrono::seconds{2};
80 std::this_thread::sleep_until(clock);
81 std::chrono::duration<double> elapsed = finish - start;
82 std::cout << ".\t\tbetween " << chunk_id * chunk_size * WINDOW_SIZE
83     << " and " << (chunk_id + 1) * chunk_size * WINDOW_SIZE << "\t"
84     << "\ttime: " << elapsed.count() << "s\t"
85     << "packet_count: " << packet_count << "\n";
86 if (elapsed.count() > 0.95) {
87     std::cout << "Unexpected delay detected! Redoing chunk!";
88     --chunk_id;
89 }
90
91 //WRITE CODE HERE
```

Figure 9: sequence_inference.cpp Step 1

Task Set 4: Acknowledgement Number Inference

Objective

Infer the next expected acknowledgement number from the server such that the Attacker's packets are acknowledged in the proper order.

This task set will be conducted in the ack_inference.cpp file. Use the following command to open the correct file:

```
$ gedit ack_inference.cpp
```

Task 4.1: Determine the Correct Acknowledgement Number

To find the next expected acknowledgement number of the server, a multi-bin search is used. The maximum window size for acknowledgement num-

bers is 2^{30} . This is defined in the code as *one_G*. By sending one packet to the 0 sequence number, two to *one_G* - 1, four to *two_G* - 1, and eight to *three_G* - 1, the amount of packets received can be counted to find the area in which the correct number lies. For instance, if 94 challenges are received, then it indicates that challenges from *one_G* and *two_G* have been triggered, thus the range is in between the two of them. If 96 challenges are observed, then it indicates that challenges from only *two_G* have been triggered, thus the range is inside of *two_G*.

Action

4.1.1 - Write code to handle missing four packets on line 89, eight packets on line 100, and 12 packets on line 111 of *ack_inference.cpp* as seen in Figure 10 below.

```

86         case (ACK_LIMIT - 4):
87
88
89             // WRITE CODE HERE:
90
91
92         break;
93     case (ACK_LIMIT - 6):
94         left_ack = 0;
95         right_ack = 2 * one_G - 1;
96         break;
97     case (ACK_LIMIT - 8):
98
99
100         // WRITE CODE HERE:
101
102
103         break;
104     case (ACK_LIMIT - 9):
105         left_ack = 2 * one_G - 1;
106         right_ack = 4 * (one_G - 1);
107         break;
108     case (ACK_LIMIT - 12):
109
110
111         // WRITE CODE HERE:
112
113

```

Figure 10: *ack_inference.cpp*

After altering the code, compile it by navigating to the *attacker_cpp* folder

through the terminal, removing the exploit.exe file, and then entering the make command.

Once this is completed the left boundary of the acceptable ACK range can be found and spoofed packets can be sent to the client.

If all the code is working correctly, you should be able to write messages into the console that will show up as the server in the client's messaging terminal.

Submission

Submit a lab report using the provided [lab report template](#). Create a lab report that consists of text and images from each task in the lab. Describe the process you went through to solve each task. Be sure to include how you solved the game theory table. With your submission include the synchronize_clock.cpp, 4tuple_inference.cpp, sequence_inference.cpp, and ack_inference.cpp files.

Summary

In this lab you've learned what an Off-Path TCP attack is, how it's implemented, and what an attacker can do with it.

Off-Path TCP attacks are of particular concern for public networks since the networks are inherently less secure. As a consequence of this, Off-Path TCP attacks are far easier for an attacker to perform on public networks so, knowing how this attack works and is performed will be beneficial for determining appropriate defensive measures.

References

- [1] Yue Cao, Zhiyun Qian, Zhongjie Wang, Tuan Dao, Srikanth V. Krishnamurthy, and Lisa M. Marvel. Off-Path TCP exploits: Global rate limit considered dangerous. In *25th USENIX Security Symposium (USENIX*

Security 16), pages 209–225, Austin, TX, August 2016. USENIX Association.